

Formato di file YAML

YAML (Yet Another Markup Language o YAML Ain't Markup Language) è un formato di serializzazione di dati leggibile dagli esseri umani. È molto usato per configurazioni perché è semplice e ordinato.

Idea di base: YAML è fatto di **chiavi e valori** organizzati in una struttura gerarchica con indentazione. L'indentazione è fatta di **SPAZI, MAI TAB**. È simile a JSON, ma più leggibile, dicono.

YAML ha diverse regole:

1. Struttura base: oggetto composto da chiave e valore

- Ogni chiave è seguita da `:` e poi dal valore.
- Non servono apicetti se il valore è semplice. Se il valore è composto da più parole occorre circondarlo con apici.
- I valori possono essere stringhe, numeri, booleani o oggetti (annidati).
- I valori possono anche essere multipli, nel qual caso si parla di liste ed allora ogni elemento della lista sta in una riga diversa ed è preceduta da un carattere `-` (meno). Gli elementi della lista devono essere indentati rispetto alla chiave, usando solo gli SPAZI, NON i TAB:
- I valori possono essere oggetti, quindi possiamo avere oggetti annidati. Gli oggetti devono essere indentati rispetto alla chiave.
- Una mappa (o dizionario) in YAML è un insieme di chiavi e valori, dove ogni chiave ha un valore associato. Le mappe sono rappresentate da coppie chiave-valore indentate. Tutti gli oggetti ad uno stesso livello sono indentati nello stesso modo.

2. Esempi di coppie chiave: valore

```
chiave: valore
numero: 42
booleano: true
stringa: "Testo con spazi"
```

3. Esempio di liste. Ogni elemento della lista inizia con `-` e va indentato.

```
ports:
  - "5000:5000"
  - "8888:80"
  - "10022:22"
```

4. Esempio di oggetti multipli (mappe o dizionari) e annidati. Si rappresentano con indentazione (SPAZI, **MAI TAB**).

```
services:
  web:
    image: nginx:latest
    depends_on:
      - app
  app:
    build: ./app
    ports:
```

5. Modo compatto per liste di oggetti

Si può scrivere una lista di oggetti in linea usando la coppia {} per ciascun oggetto della lista

```
persone:
- { nome: Luca, età: 28 }
- { nome: Laura, età: 32 }
```

6. Multi-line stringhe. Si può scrivere un valore distribuendolo su più righe per comodità di visualizzazione o proprio per inserire delle andate a capo nel valore. Nel primo caso occorre specificare il simbolo > mentre nel secondo caso si usa il simbolo |

- > indica che la stringa continua su più righe, ma senza newline
- | mantiene i newline

```
descrizione: >
  Questa è una stringa
  multilinea YAML che viene
  trattata come una sola linea.
```

```
testo: |
  Questa è una stringa multilinea
  che mantiene i newline.
```

7. Alias (anchor) e riferimenti. YAML permette di definire alias per evitare ripetizioni. La definizione dell'alias definisce **un valore** (semplice o una lista o un oggetto) a cui ci si può riferire in un punto del file yaml per usare una copia di quel valore. Nella definizione, il nome dell'alias viene specificato con & mentre il punto in cui si vuole usare l'alias si indica con *

Il nome assegnato all'alias sta tra la chiave ed il valore che assegno all'alias.

```
luogo_originale: &luogo Milano
nome_chiave: nome_alias valore
```

L'alias (*nome) rappresenta **solo un valore** (che può essere semplice o complesso).

L'alias può essere associato a **qualsiasi chiave** e a questa fornisce il valore.

L'alias **NON** può essere usato **al posto del nome della chiave**.

Se l'alias è un valore complesso, allora per usarlo devo utilizzare una notazione di Yaml detta "merge key" <<: *anchor che serve per unire oggetti

Esempio di alias di un valore semplice.

```
luogo_originale: &luogo Milano <--definisco alias luogo di valore Milano
ufficio: *luogo
residenza: *luogo
```

che viene visto come:

```
luogo_originale: Milano
ufficio: Milano
residenza: Milano
```

Esempio di alias di un oggetto complesso

```

persona_base: &persona      <-- salvo l'intero valore come anchor
  nome: Luca
  città: Milano
impiegato:
  <<: *persona
  ruolo: Sviluppatore
cliente:
  <<: *persona
  codice_cliente: 1234

```

che viene visto come:

```

persona_base:
  nome: Luca
  città: Milano
impiegato:
  nome: Luca
  città: Milano
  ruolo: Sviluppatore      <-- notare che ho aggiunto un oggetto ruolo
cliente:
  nome: Luca
  città: Milano
codice_cliente: 1234      <-- notare che ho aggiunto un oggetto codice

```

Esempio di alias di una lista:

Definiamo una lista una volta e poi la usiamo più volte:

```

lingue_comuni: &lingue
- italiano
- inglese
studente:
  lingue_parlate: *lingue
insegnante:
  lingue_insegnate: *lingue

```

viene espanso così:

```

studente:
  lingue_parlate:
    - italiano
    - inglese
insegnante:
  lingue_insegnate:
    - italiano
    - inglese

```

```

valore_comune: &default      <-- definisco l'alias di nome default

```

Compose file

Un "compose file" è un file YAML utilizzato da Docker Compose per definire e gestire applicazioni multi-container.

Il formato di un compose file richiede specificatamente che alcune chiavi abbiano come valore delle mappe (insieme di oggetti) ed altre abbiano come valore delle liste (quelle che iniziano con il meno -). Bisogna porre attenzione alla sintassi.

Un compose file è suddiviso in diverse sezioni, tra cui:

- **version:** specifica la versione del formato del file Compose.
- **services:** definisce i servizi che compongono l'applicazione.
- **volumes:** definisce i volumi per la persistenza dei dati.
- **networks:** definisce le reti per la comunicazione tra i servizi.
- **secrets:** stabilisce come fornire informazioni riservate ai container.

Esempio 2: schema (non completo) di applicazione con database

Questo esempio aggiunge un database MySQL all'applicazione web.

```
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - app
  app:
    build: ./app
    ports:
      - "5000:5000"
    environment:
      MYSQL_HOST: db
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

Spiegazione:

- Il servizio `db` utilizza l'immagine MySQL 8.0 e definisce una variabile d'ambiente per la password di root.

- `volumes` definisce un volume denominato "db_data" per la persistenza dei dati del database.
- L'environment del servizio app, specifica che il servizio potrà raggiungere il database mediante il nome del servizio "db".

Esempio 3: schema di utilizzo delle reti

Questo esempio definisce una rete personalizzata per la comunicazione tra i servizi.

```
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - app
    networks:
      - my_network
  app:
    build: ./app
    ports:
      - "5000:5000"
    environment:
      MYSQL_HOST: db
    networks:
      - my_network
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - my_network
volumes:
  db_data:
networks:
  my_network:
```

Spiegazione:

- La sezione `networks` definisce una rete denominata `my_network`.
- Ogni servizio è collegato alla rete `my_network`, consentendo la comunicazione tra di essi. La rete non è dichiarata privata, quindi consente di comunicare con l'host.

Comandi Docker Compose utili

- `docker compose build`: costruisce le immagini definite nel compose file.
- `docker compose up`: avvia i servizi definiti nel compose file.
- `docker compose down`: arresta e rimuove i servizi.
- `docker compose ps`: elenca i servizi in esecuzione.
- `docker compose logs`: visualizza i log dei servizi.